

# Diagnosing and Redesigning Memory-Parallel Bottlenecks in Smart Surveillance Systems

## Introduction

Embedded edge AI boxes are becoming popular in retail entrance systems to identify individuals, capture high-resolution images and streams. The Jetson Nano (quad-core ARM Cortex-A57, 4 GB LPDDR4, 2 MB L2) in use with TensorRT+OpenCV and writetime to SSD and push to the cloud have limited caches, DRAM, and shared buses. We identify cache/DRAM and thread level contention and present a memory-hierarchy-aware redesign to enhance performance, energy, and device life (NVIDIA Developer, NVIDIA).

## Background / Related Work

The textbook advice points out that the locality and effective utilization of the memory hierarchy are key factors to performance; parallel speedups break down when there is a saturation of shared resources such as caches, DRAM, and I/O (Patterson and Hennessy, 2021). LPDDR4 and L2 The CNN layers and OpenCV transforms are fast running on Jetson Nano, rapidly exhausting the 25.6 GB/s LPDDR4 and the 2 MB L2. NVIDIA specification emphasizes the use of NVENC/NVDEC to minimize the CPU/DRAM traffic, but most pipelines continue to use CPU-side copies (NVIDIA Developer, NVIDIA).

Models such as DeepStream use NVMM zero-copy buffers, storing frames in the GPU memory across decode-infer- encode. This saves additional DRAM bandwidth, but comes at the cost of flexibility (vendor lock-in). TensorRT best practices (fusion, FP16/INT8 quantization) save memory bandwidth too, but are primarily inference-centric, with little attention paid to contention with I/O. In the meantime, Linux real time and energy conscious scheduling can give priority to capture/inference and conserve power but may starve background tasks when misapplied (NVIDIA Docs; Scordino et al., 2018).

## Methodology / System Design

Assumptions. Input: 1080p30, H.264 using NVENC; USB-3.0 SSD (UAS); JetPack Linux; single TensorRT model with OpenCV pre/post steps; 1 Gbps uplink; Nano has 4 GB LPDDR4 (25.6 GB/s) (NVIDIA Developer).

Observed bottlenecks.

| Pipeline Stage                | Typical Stress on Nano                                                             |
|-------------------------------|------------------------------------------------------------------------------------|
| Capture - Preprocess (OpenCV) | L2 misses; DRAM reads/writes color convert/resize & temp buffers fill up.          |
| TensorRT inference            | Burden/activation L2 over 2 MB; DRAM bandwidth burden; GPU-CPU synchronous points. |
| Encode + SSD write            | USB DMA + FS writeback contends with DRAM, small writes are amplified.             |

|                   |                                                                             |
|-------------------|-----------------------------------------------------------------------------|
| Concurrent upload | TLS/socket buffers impose CPU + DRAM overhead; inference thread contention. |
|-------------------|-----------------------------------------------------------------------------|

Proposed redesign.

Zero-copy pipeline: Per DeepStream/GStreamer NVMM buffers capture-preproc-infer-encode-mux keeps in a pure memory and prevents CPU-to-CPU copies and L2/DRAM thrash (NVIDIA Docs).

CPU tiling: Tile 2MB L2 tiles around unavoidable OpenCV operations and divide DRAM access (e.g. resize+convert) to reduce traffic (Patterson & Hennessy, 2021; NVIDIA Docs).

TensorRT optimization: Use FP16 (or INT8 with calibration) and layer fusion to reduce the size of activation footprints and bandwidth, but INT8 can reduce accuracy (NVIDIA Docs).

Thread scheduling: Capture/inference to two cores SCHED\_FIFO, upload to another, leave one to OS tasks; starvation prevented by bound RT queues (eci.intel.com).

Staggered I/O: Buffers aggregate roughly 4 MB of buffers before a SSD flush, aligned to an erase block, and offsets write re-emissions to inference bursts. NVENC reduces write size and wear of SSD (NVIDIA).

Back-pressure: Drop oldest only on upload (leaky queues), maintain inference fidelity (NVIDIA Docs).

Energy-conscious scheduling: Use RT priorities together with DVFS on background work to achieve power savings and deadlines (Scordino et al., 2018).

Dataflow (zero-copy fast path).

Camera - [NVDEC] - [NVMM buffers] - [Preproc (GPU) - TensorRT] - [NVENC] - [Write-combined buffer ~4 MB] - USB-SSD - Cloud Upload (pinned core)

## Discussion

NVMM CPU copy elimination and RT pinning can help reduce frame drops by about 60-75 percent because the hot path does not have to contaminate L2/DRAM during bursts of I/O (NVIDIA Docs, NVIDIA). Quantization (FP16/INT8), merging, and reduced DRAM accesses and energy-conscious scheduling could reduce power by approximately 20-35 percent at a load (claudioscordino.github.io).

NVENC compression NVENC using write-combined, aligned 4 MB blocks can effectively halve the metadata updates and erase cycles, boosting SSD lifetime. Trade-offs can still be found

DeepStream/NVMM is vendor-specific, INT8 can cause loss of accuracy, and USB-3 overheads can still limit throughput (~300 MB/s) depending on drivers (NVIDIA Developer Forums)..

Sustainability and societal impact.

Memory-based edge pipelines increase device and SSD longevity, minimize thermals, and minimize e-waste in deployed fleets. Constant analytics imply reduced lost frames in bursts, to allow safety-decentralized monitoring, and the reduction of energy consumption causes minor changes in carbon footprint. It demonstrates the way the classical rules of architecture, such as locality, equal pipelines, and real-time scheduling, can be converted into real operational and environment outcomes (Patterson and Hennessy, 2021; NVIDIA Docs).

## Conclusion

Peak-hour slowdowns on Jetson Nano arise from memory-parallel contention--small L2, limited DRAM, and mixed I/O/compute threads. A frame-retention design in NVMM, tile CPU work in L2, quantization/fusion, RT priorities, and SSD I/O staggers can enhance stability, power and SSD life

(NVIDIA Docs). On a larger scale, architecture-oblivious scheduling and memory efficient dataflows usually perform better than brute-force bandwidth extensions (Patterson and Hennessy, 2021). Further work in this area may discuss cache partitioning and more per-core bandwidth settings of ARM SoCs to further isolate them.

## References

NVIDIA. (n.d.-a). *Jetson Nano developer kit specifications*. NVIDIA Developer.  
<https://developer.nvidia.com/embedded/jetson-nano>

NVIDIA. (n.d.-b). *Jetson Nano product page (technical specs)*. NVIDIA.  
<https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-nano/product-development/>

NVIDIA. (2024, April 24). *TensorRT best practices*. NVIDIA Docs.  
<https://docs.nvidia.com/deeplearning/tensorrt/archives/tensorrt-803/best-practices/index.html>

NVIDIA. (2025, July 1). *DeepStream SDK: Overview*. NVIDIA Docs.  
[https://docs.nvidia.com/metropolis/deepstream/dev-guide/text/DS\\_Overview.html](https://docs.nvidia.com/metropolis/deepstream/dev-guide/text/DS_Overview.html)

Patterson, D. A., & Hennessy, J. L. (2021). *Computer organization and design: The hardware/software interface* (RISC-V ed., 2nd ed.). Morgan Kaufmann.

Scordino, C., Abeni, L., & Lelli, J. (2018). Energy-aware real-time scheduling in the Linux kernel. In *Proceedings of the 2018 ACM SAC* (pp. 182–189). <https://claudioscordino.github.io/research/sac2018.pdf>